

What an agent is in this course

Claude Certified Developer, Foundation (CCDV-F) · UST masterclass

EN

Session 5 of 12 · English notes

A long context-window question (“this generation has been running forever — how is context managed?”) is what pushed session 5 into the course’s working definition of an **agent**. Code generation is not one LLM call. It is an **agentic loop**: a main agent (orchestrator) writes a to-do list; **sub-agents** execute slices with their own context windows, prompts, tools, memory, skills, and optionally their own model and permission mode. Claude Code itself is the reference agentic client; the Foundation exam later asks you to build similar systems with Claude APIs.

DOCS AND MARKETPLACE LISTS CHANGE

Built-in sub-agents (Explore, Plan, status-line helpers) and the official tool list (Glob, Grep, Read, Write, ...) are rearranged in documentation. The trainer had to hunt a vertical tool list live. Use **Claude Code**, **tool use**, and **Managed Agents** for current names.

Main agent versus sub-agent

In Claude Code there is one **main agent**: the session where you type. It orchestrates and plans. A **sub-agent** executes a task. The trainer’s wording: “agent is nothing but a sub-agent in the case of Claude” once you are talking about the specialised markdown files you add. There is one main agent and many sub-agents. Sub-agents can auto-trigger when a prompt matches their *description*, or you invoke them. They each run in their own context window with a custom system prompt, their own tool access, and independent permissions.

Piece	Main agent (Claude Code session)	Sub-agent
Job	Orchestrate: memory + skills + context → to-do list	Execute one slice (folders, backend, frontend, Docker, review)
Context	The chat you see; can grow large	Fresh window scoped to the slice
Memory files	Global CLAUDE.md always in play	Global plus the folder CLAUDE.md that matches the task (backend vs frontend)
Short-term scratch	Sandbox; discarded when the slice finishes; you do not control it	Same idea, local to that agent run

Long-term memory is files, not the chat

Rajesh asked whether “memory” lived in the sandbox with a short TTL. The trainer split: CLAUDE . md files in the git repo are **long-term** and physical (global, backend/CLAUDE . md, frontend/CLAUDE . md).

Sandbox state that stitches function-by-function generation is **short-term** and discarded. CLAUDE.md is “master configuration, not a master agent.” Nested files do not spawn extra orchestrators; they are which instruction packet a given agent loads.

Three agents on a full-stack job can therefore run with three instruction mixes: (1) global only (create folders), (2) global + backend, (3) global + frontend. They are not one huge message. That is how a hours-long generation stays inside context limits: bits and pieces, planned, then sent.

Four configurations every agent has

Whether main or sub, the trainer listed four always-on ingredients, then extras you pin in the agent file:

1. Prompt (the agent's system / role text).
2. Context (what is in the window for this run).
3. Tools (built-in and/or MCP — e.g. ECR instead of Docker Hub for base images).
4. Memory and skills (which CLAUDE.md and which SKILL.md packets).
5. Optional: model, permission mode, max turns, disallowed tools, hooks, isolation, colour, initial prompt, effort, background.

Required fields when you author a sub-agent: **name** and **description**. Description is how the orchestrator routes (“reviews code for quality... use immediately after writing or modifying code”). Tools can be a tight list (Read only → reviewer cannot Write) or a wildcard if you truly do not know. Custom tools are MCP tools, not a random local function pretending to be a tool (that clarification was explicit).

Where agent files live

- Project: a `.claude/agents` (trainer: “dot cloud slash agents”) folder in the repo — cloned with the project.
- User profile (Windows / macOS / Linux): available to all that user's projects.
- Plugins: bundled with hooks and skills (Module 06).
- CLI flags if you launch agents from the command line.
- Built-ins: Explore (filesystem-oriented tool use), Plan, status-line setup, and whatever the current docs list — used automatically.

Stateless models, stateful orchestration

A learner hoped CLAUDE.md and skills would be “cached” like repetitive prompt words. The trainer: LLMs are **stateless**; one request may hit a different instance (horizontal scaling). Prompt cache is a later, explicit API feature — not something you assume in Claude Code. Tool *results* can be cached in shared infrastructure (he mentioned Redis-like caches) and reused across instances. Chat history is held by the *client* / *orchestrator*, which is why you can resume a to-do list. Multiple sub-agents on one project still coordinate through that orchestrator, not by chatting with each other as peers unless the main agent passes artefacts (files, OpenAPI, types).

Exam destination

“Implement something like Claude Code” using agent SDK / Claude APIs is the Foundation north star. In this module you only need the vocabulary: orchestrator, sub-agent, own context window, description-based delegation, tool allow/deny, inherit vs pin model, plan editable then frozen. Too many overlapping CLAUDE.md files can hallucinate — Pedro’s warning; the mitigation taught is to force a clarify step in global memory before generating.

Official documentation

- **Claude Code overview**

<https://code.claude.com/docs/en/overview>

Sub-agents, built-in agents, and project vs user scope.

- **Managed Agents quickstart**

<https://platform.claude.com/docs/en/managed-agents/quickstart>

Platform-side agents you will meet when you leave the IDE client.

- **Tool use overview**

<https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>

Read/Write/Grep-style tools and MCP tools on an agent.

- **MCP**

<https://docs.claude.com/en/docs/agents-and-tools/mcp>

How an agent is limited to ECR vs Docker Hub in the Docker example.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic’s current docs over session anecdotes. Not official exam material.